

RepoExplorer

AI-Powered Codebase Understanding Tool

Full Technical Documentation

Sahil Arun Dhawane

Vishwakarma Institute of Technology, Pune
B.Tech Electronics & Telecommunications

Repository: github.com/Sahil0282/repo-explorer

Prepared for placement interview preparation

April 2026

Contents

1	The Problem: Why This Tool Exists	4
1.1	The Core Pain Point	4
1.2	Why Existing Solutions Fall Short	4
1.2.1	GitHub's Built-in Navigation	4
1.2.2	IDE Tools like Copilot and Cursor	4
1.2.3	Documentation	4
1.3	The Gap RepoExplorer Fills	4
1.4	Concrete Example of the Problem and Solution	5
2	System Overview: How RepoExplorer Solves This	6
2.1	High-Level Architecture	6
3	The AI Service: Every Component Explained	8
3.1	The RAG (Retrieval-Augmented Generation) Paradigm	8
3.1.1	What is RAG and Why We Use It	8
3.2	Embedding Model: MiniLM-L6-v2	8
3.2.1	What is an Embedding?	8
3.2.2	Why MiniLM-L6-v2 Specifically?	8
3.2.3	How Embedding Works in Code	9
3.3	Vector Database: ChromaDB	9
3.3.1	What is a Vector Database?	9
3.3.2	Why ChromaDB?	9
3.3.3	Caching Strategy	10
3.4	Chunking Strategy: What We Index and How	10
3.4.1	Naive Chunking (What We Avoided)	10
3.4.2	AST-Based Function-Level Chunking (What We Use)	10
3.4.3	Upgrade 1: Enriched Chunks	11
3.4.4	Upgrade 2: File Summary Chunks	11
3.5	Hybrid Retrieval: BM25 + Vector Search + RRF	11
3.5.1	Why Pure Vector Search is Not Enough	11
3.5.2	BM25: Keyword Search	12
3.5.3	Reciprocal Rank Fusion (RRF)	12
3.5.4	Why Hybrid Search Outperforms Either Alone	12
3.6	The LLM: Google Gemini 2.5 Flash	12
3.6.1	Why Gemini 2.5 Flash?	12
3.6.2	Why Not GPT-4?	12
3.6.3	The Prompt Engineering	13
3.7	The Complete Indexing Pipeline	13
3.8	The Complete Query Pipeline	14
4	Node.js Backend: Detailed Breakdown	15
4.1	Key Libraries and Their Roles	15
4.1.1	Express.js	15
4.1.2	simple-git	15
4.1.3	@babel/parser and @babel/traverse	15
4.1.4	axios	15
4.1.5	mongoose and MongoDB	16
4.1.6	cors	16
4.1.7	dotenv	16

4.2	API Endpoints	16
4.2.1	POST /api/repo/analyze	16
4.2.2	POST /api/repo/query	16
4.2.3	GET /api/repo/file	16
4.2.4	POST /api/repo/evolution [NEW]	17
4.3	The File Tree Builder (fileTree.js)	17
4.4	The Function Extractor (extractFunctions.js)	17
4.5	The Git Evolution Tracker (gitEvolution.js) [NEW]	18
5	Python AI Service: File-by-File Breakdown	19
5.1	embedder.py	19
5.2	vector_store.py	19
5.3	llm.py	19
5.4	main.py	20
6	Frontend: React Application	21
6.1	Technology Stack	21
6.2	Pages	21
6.2.1	LandingPage.jsx	21
6.2.2	ChatPage.jsx	21
6.3	Components	22
6.3.1	ChatMessage.jsx	22
6.3.2	FileTree.jsx	22
6.3.3	FileViewer.jsx	22
6.3.4	FocusMap.jsx	23
6.3.5	RepoHeader.jsx	23
7	Phase-Wise Development: What Was Built, Issues Found, How Fixed	24
7.1	Phase 1: Core Pipeline — Clone, Parse, Index	24
7.1.1	What was built	24
7.1.2	Issues Found	24
7.1.3	Result at End of Phase 1	25
7.2	Phase 2: RAG Quality Improvements	25
7.2.1	Problem Identified	25
7.2.2	Upgrade 1: Enriched Chunks	25
7.2.3	Upgrade 2: File Summary Chunks	25
7.2.4	Upgrade 3: Hybrid Search (BM25 + Vector + RRF)	25
7.2.5	Test File Filtering	25
7.2.6	Result at End of Phase 2	25
7.3	Phase 3: Prompt Engineering	26
7.3.1	Problem Identified	26
7.3.2	Solution	26
7.3.3	Result	26
7.4	Phase 4: Frontend Development	26
7.4.1	Landing Page	26
7.4.2	Chat Page	26
7.4.3	File Tree Component	26
7.4.4	File Viewer Component	26
7.4.5	Focus Map Component	26
7.4.6	Issues During Frontend Development	27
7.5	Phase 5: GitHub and Documentation	27

7.6	Phase 6: Git Evolution Feature [NEW]	27
7.6.1	Motivation	27
7.6.2	What Was Built	27
7.6.3	Issues During Phase 6	28
7.6.4	Result at End of Phase 6	28
8	Current Limitations of the MVP	29
8.1	Repository Constraints	29
8.2	RAG Quality Limitations	29
8.3	Infrastructure Limitations	29
9	Ongoing and Future Improvements	30
9.1	Improvement 1: Better Anonymous Function Naming (High Priority)	30
9.2	Improvement 2: Python File Support	30
9.3	Improvement 3: Call Graph / Dependency Tracking	31
9.4	Improvement 4: BM25 Index Caching	31
9.5	Improvement 5: Session Persistence	31
9.6	Improvement 6: Deployment	32
9.7	Improvement 7: Clickable File Paths in Chat	32
9.8	Improvement 8: Rate Limiting	32
9.9	Improvement 9: Migrate Gemini SDK to google-genai [NEW]	33
9.10	Improvement 10: Fix requirements.txt [NEW]	33
10	Interview Preparation: Questions You Will Be Asked	34
10.1	System Design Questions	34
10.2	AI/ML Questions	34
10.3	System-Specific Questions	35
11	Complete Tech Stack Reference	36

The Problem: Why This Tool Exists

The Core Pain Point

Every software developer has faced this situation: you are assigned to fix a bug or add a feature in a codebase you have never seen before. Or you want to use an open-source library and need to understand how it works internally. Or you join a new team and are handed a repository with hundreds of files and zero documentation.

In all of these cases, the process of understanding the codebase is brutally slow and frustrating. You open the repository on GitHub and see this:

Example Scenario: You need to understand how authentication works in a Node.js backend project. The repository has 200 files across 15 folders. You have no idea where to start. Is it in `middleware/auth.js`? Or `utils/jwt.js`? Or `controllers/userController.js`? Or is it split across all three? You have to manually open each file, read through hundreds of lines of code, and slowly build a mental map of how everything connects. This process can take hours or even days.

Why Existing Solutions Fall Short

GitHub's Built-in Navigation

GitHub lets you browse files and view code. But it only shows you *what* the code looks like. It cannot tell you *how* it works, *why* it's structured this way, or *which files you should look at* to understand a specific feature. You still need to already know which file to open.

IDE Tools like Copilot and Cursor

Tools like GitHub Copilot Chat and Cursor are excellent for developers who are *already working inside* a codebase. They require you to:

- Clone the repository to your local machine
- Open it in a supported IDE (VS Code, JetBrains, etc.)
- Wait for indexing to complete
- Have the right extensions installed

This is a 15–30 minute setup process minimum. And it only works if you *own* the codebase or have local access to it.

Documentation

Most codebases have little to no documentation. Even when documentation exists, it is often outdated and does not reflect the current state of the code.

The Gap RepoExplorer Fills

RepoExplorer targets a fundamentally different user moment:

The user does not own the code. They are outside the codebase, looking in.

Examples of this user:

- A new joiner who has not been given laptop access yet
- An open-source contributor evaluating whether a library suits their needs
- A tech lead reviewing a candidate's GitHub project during an interview
- A developer asked to debug someone else's project without any context
- A student trying to understand how a popular open-source project is structured before contributing

For all of these people, the existing tools are useless. RepoExplorer solves this by allowing any user to paste a GitHub URL, ask a question in plain English, and receive a structured answer with exact file citations and a visual map of which functions are involved — all in under 60 seconds with zero local setup.

Concrete Example of the Problem and Solution

Without RepoExplorer:

User wants to understand how authentication works in `expressjs/express`. They open GitHub, browse to the repository, see 213 files, start opening random files, spend 45 minutes reading code, still are not sure which middleware runs first.

With RepoExplorer:

User pastes `https://github.com/expressjs/express` into the tool. Asks: *“how does authentication work in this project?”*. Within 5 seconds receives:

Authentication in this project involves the following flow:

1. **examples/auth/index.js (lines 25–34):** The `loadUser` middleware fetches a user based on `req.params.id` and attaches it to `req.user...`
 2. **controller/authController.js (lines 56–64):** Upon successful sign-up, an `authtoken` is set as an HTTP cookie and included in the JSON response...
- ...and so on with exact citations.

The user also sees a Focus Map on the right side showing exactly which files and functions were involved, with the ability to hover over any function to see its exact code.

System Overview: How RepoExplorer Solves This

High-Level Architecture

RepoExplorer is built as three separate services that work together:

1. **Frontend** — React application (runs on port 5173)
2. **Node.js Backend** — Express server (runs on port 8000)
3. **Python AI Service** — FastAPI server (runs on port 8001)

Flow when a user analyzes a repository:

```
User (Browser) → React Frontend → POST /api/repo/analyze (Node.js:8000)
→ Clone repo from GitHub (simple-git)
→ Parse all JS/TS files (Babel AST)
→ Extract all functions with code + metadata
→ Compute git evolution map (gitEvolution.js) [NEW]
→ POST /index (Python FastAPI:8001)
→ Build enriched chunks
→ Generate embeddings (MiniLM-L6-v2)
→ Store in ChromaDB
→ Return file tree + stats to Frontend
```

Flow when a user asks a question:

```
User types question → POST /api/repo/query (Node.js:8000)
→ POST /query (Python FastAPI:8001)
→ Hybrid Search: BM25 + Vector + RRF
→ Top 5-8 relevant chunks retrieved
→ Chunks + question sent to Gemini 2.5 Flash
→ Structured answer returned
→ Answer + source files + source functions sent to Frontend
→ Chat message rendered with markdown
→ Focus Map updated with relevant file/function nodes
```

Flow when a user requests function evolution history: [NEW]

```
User requests evolution → POST /api/repo/evolution (Node.js:8000)
→ gitEvolution.js reads last 20 commits (git show --unified=0)
→ Parse added/deleted lines per commit with line numbers
→ Map line changes → extracted functions by startLine/endLine range
→ Build evolutionMap keyed as filePath::functionName
→ POST /evolution-summary (Python FastAPI:8001)
→ Gemini summarizes change history per function
→ Structured evolution timeline returned to Frontend
```

Important distinction: Git commit diffs are **not embedded**. They are sent directly to Gemini as text for summarization. Only code chunks (function bodies + file summaries) go through MiniLM embeddings and ChromaDB. The two pipelines are completely

independent.

The AI Service: Every Component Explained

This is the heart of the system. The Python FastAPI service handles all AI-related operations. It lives in the `service/` folder.

The RAG (Retrieval-Augmented Generation) Paradigm

What is RAG and Why We Use It

Large Language Models like Gemini are trained on massive datasets but have two fundamental limitations:

1. **Knowledge cutoff:** They don't know about code written after their training cutoff.
2. **Context window:** You cannot feed an entire codebase (potentially millions of lines) into an LLM prompt — it has a token limit.

RAG solves both problems. Instead of sending the entire codebase to the LLM, you:

1. **Index phase:** Break the codebase into small chunks, convert each chunk to a vector (embedding), store in a vector database.
2. **Query phase:** When a question arrives, find the most relevant chunks using similarity search, send only those chunks to the LLM.

The LLM then answers based on the retrieved chunks — it is grounded in real code, not hallucinating. This is the core principle behind RepoExplorer.

Embedding Model: MiniLM-L6-v2

What is an Embedding?

An embedding is a numerical representation of text as a vector (list of floating point numbers). Text with similar meaning gets vectors that are mathematically close to each other in high-dimensional space.

Example:

- "function that handles user authentication" → [0.23, -0.45, 0.12, ...]
- "JWT token verification middleware" → [0.21, -0.43, 0.15, ...]
- "sorting a list of integers" → [-0.67, 0.89, -0.34, ...]

The first two vectors are close together (similar meaning), the third is far away.

Why MiniLM-L6-v2 Specifically?

We use the `sentence-transformers/all-MiniLM-L6-v2` model from Hugging Face. Here is why this model was chosen over alternatives:

Model	Size	Dimensions	Quality	Cost
OpenAI text-embedding-3-small	API-only	1536	Excellent	Per API call
OpenAI text-embedding-ada-002	API-only	1536	Good	Per API call
all-MiniLM-L6-v2	90 MB	384	Good	Free, local
all-mpnet-base-v2	420 MB	768	Better	Free, local

MiniLM-L6-v2 was chosen because:

- **Zero API cost** — runs entirely on CPU, no external API calls for embeddings
- **Small size** — only 90 MB, downloads once and cached locally
- **Fast** — can embed 3000+ functions in under 30 seconds on a MacBook
- **Good quality** — 384-dimensional vectors, sufficient for code retrieval
- **Student project constraint** — OpenAI embeddings cost money; this is free

How Embedding Works in Code

The `embedder.py` file loads the model once at startup:

```
1 from sentence_transformers import SentenceTransformer
2
3 # loads 90MB model, cached after first download
4 model = SentenceTransformer('all-MiniLM-L6-v2')
5
6 def embed_text(text: str) -> list[float]:
7     return model.encode(text).tolist() # returns list of 384 floats
8
9 def embed_batch(texts: list[str]) -> list[list[float]]:
10    # batch processing - much faster than one at a time
11    return model.encode(texts, batch_size=64, show_progress_bar=True).tolist()
```

The model loads once at service startup. Every query and every chunk uses the same model. **This is critical** — you must use the same model for both indexing and querying, otherwise the vectors are not comparable.

Vector Database: ChromaDB

What is a Vector Database?

A vector database stores embedding vectors alongside their original text and metadata. Its primary operation is **similarity search**: given a query vector, find the stored vectors that are most mathematically similar (using cosine similarity or L2 distance).

Why ChromaDB?

ChromaDB was chosen over alternatives like Pinecone, Weaviate, or FAISS for these reasons:

- **Local persistent storage** — data saved to disk in `./chroma_store/`, survives service restarts
- **No cloud account needed** — Pinecone requires signup and has limits on free tier
- **Simple Python API** — minimal boilerplate

- **Collection-based** — each repository gets its own isolated collection

Caching Strategy

Once a repository is indexed, its ChromaDB collection persists on disk. The next time the same repository URL is analyzed, the system detects the existing collection and skips re-indexing. This means:

- First analysis of `expressjs/express`: 45–90 seconds (clone + parse + embed)
- All subsequent analyses: under 2 seconds (use cached collection)

Chunking Strategy: What We Index and How

This is one of the most important design decisions in a RAG system. How you break content into chunks directly determines retrieval quality.

Naive Chunking (What We Avoided)

The simplest approach is to split files by character count (e.g., every 500 characters is one chunk). This is bad for code because:

- A function might be split in half between two chunks
- No semantic meaning is preserved
- The chunk has no context about which file it came from

AST-Based Function-Level Chunking (What We Use)

We parse every JavaScript/TypeScript file using Babel's Abstract Syntax Tree (AST) parser and extract complete functions as individual chunks. Each function is one chunk — never split.

The AST is a structured tree representation of code. For example:

```
1 function greet(name) {  
2     return "hello " + name;  
3 }
```

becomes the AST node:

```
1 {  
2     "type": "FunctionDeclaration",  
3     "id": { "name": "greet" },  
4     "params": [{ "name": "name" }],  
5     "body": { ... }  
6 }
```

We use `@babel/parser` and `@babel/traverse` in Node.js to walk this tree and extract four types of functions:

1. **FunctionDeclaration** — `function foo() {}`
2. **FunctionExpression** — `const foo = function() {}`
3. **ArrowFunctionExpression** — `const foo = () => {}`
4. **ClassMethod** — methods inside ES6 classes

Upgrade 1: Enriched Chunks

A raw function chunk has poor retrieval quality because it lacks context. We upgraded to **enriched chunks** that include:

```
1 File: lib/router/index.js
2 Folder: lib/router
3 File imports: finalhandler, debug, Layer, Route
4 Other functions in this file: proto.handle, proto.use, proto.route
5 Function: next
6 Type: FunctionDeclaration
7 Lines: 136-185
8
9 Code:
10 function next(err) {
11     // ... actual function code
12 }
```

Adding the file path, folder, imports, and sibling function names dramatically improves semantic matching because the embedding model now has rich context about where this function lives in the system.

Upgrade 2: File Summary Chunks

In addition to one chunk per function, we add **one summary chunk per file**. This chunk describes the file holistically:

```
1 FILE SUMMARY
2 File: lib/router/index.js
3 File name: index.js
4 Folder: lib/router
5 Imports: finalhandler, debug, Layer, Route
6 Total functions: 12
7 Named functions: proto.handle, proto.use, proto.route, proto.
  process_params, next
8 This file is part of the lib/router module.
```

This solves a critical retrieval gap. When a user asks an architectural question like *“how does middleware flow work?”*, they get a file summary chunk that says *“this file handles middleware dispatch”* rather than a raw function chunk. File summaries are indexed alongside function chunks, giving retrieval more surface area to match against.

Hybrid Retrieval: BM25 + Vector Search + RRF

Why Pure Vector Search is Not Enough

Vector search excels at semantic similarity. If you ask *“how does the authentication flow work?”* it finds chunks about JWT tokens, session management, password hashing — even if those exact words don’t appear in the code.

But vector search has a critical weakness: **exact keyword matching**. If a user asks *“where is authenticateToken called?”* — this is a keyword question, not a semantic one. Vector search might miss the exact function because there are no semantically similar phrases in the chunk text.

BM25: Keyword Search

BM25 (Best Match 25) is a classical information retrieval algorithm used by search engines like Elasticsearch. It scores documents based on:

- **Term frequency** — how often the query word appears in the document
- **Inverse document frequency** — rare words across the corpus get higher weight
- **Document length normalization** — longer documents don't automatically score higher

BM25 excels at exact keyword matching. If you search for `authenticateToken`, BM25 will find every chunk containing that exact string, regardless of semantic similarity.

Reciprocal Rank Fusion (RRF)

Running both searches gives two ranked lists of results. We merge them using RRF:

$$\text{RRF_score}(d) = \frac{1}{k + \text{rank}_{\text{vector}}(d)} + \frac{1}{k + \text{rank}_{\text{BM25}}(d)} \quad (1)$$

Where $k = 60$ is a constant from the original RRF paper. A document appearing at rank 1 in both lists gets the highest combined score. A document only in one list still gets partial credit. This is better than simply averaging scores because it doesn't require score normalization across two different scoring systems.

Why Hybrid Search Outperforms Either Alone

- **Vector only:** Misses exact function names, variable names, error codes
- **BM25 only:** Misses semantic synonyms (“authentication” vs. “login” vs. “sign-in”)
- **Hybrid:** Gets the best of both — semantic understanding *and* keyword precision

Research consistently shows hybrid retrieval improves recall by 20–30% over pure vector search for technical queries.

The LLM: Google Gemini 2.5 Flash

Why Gemini 2.5 Flash?

We use `gemini-2.5-flash` via Google's `google-generativeai` SDK. The reasons for this choice:

- **Free tier** — Google AI Studio provides a generous free tier for `gemini-2.5-flash`, essential for a student project
- **Speed** — Flash models are optimized for low latency. Answers arrive in 3–7 seconds
- **Code understanding** — Gemini models perform well on code-related tasks
- **Large context window** — can handle the 5–8 retrieved chunks comfortably
- **Reasoning capability** — Gemini 2.5 has strong multi-step reasoning, good for tracing execution flows across multiple files

Why Not GPT-4?

GPT-4 is excellent but:

- No meaningful free tier — requires paid OpenAI account
- Higher latency on complex prompts
- For this use case (code Q&A with retrieved context), Gemini 2.5 Flash performs comparably

The Prompt Engineering

The quality of the LLM answer depends heavily on the prompt. Our final prompt structure:

```
1 prompt = f"""You are an expert code navigator helping a developer
2 understand an unfamiliar repository.
3
4 CONTEXT:
5 - You have been given {len(context_chunks)} code chunks retrieved
6   from the repository
7 - Each chunk contains a file path, function name, line numbers,
8   and actual code
9 - The chunks were selected because they are most relevant to the
10  developer's question
11
12 YOUR JOB:
13 Answer the developer's question using ONLY the provided code chunks.
14 Do not invent, assume, or infer anything not visible in the chunks.
15
16 HOW TO ANSWER:
17 1. Start with a one-sentence direct answer
18 2. Explain details - what each function does, which files, how connected
19 3. Trace execution flow - "Function A in file X calls Function B..."
20 4. Always cite file names and line numbers
21 5. Explain middleware lifecycle when relevant
22 6. End with summary of files to examine
23
24 STRICT RULES:
25 - Only state what is clearly visible in the chunks
26 - If partial info available, state what you found and what's missing
27 - Do not repeat points
28
29 RELEVANT CODE CHUNKS:
30 {context_text}
31
32 DEVELOPER'S QUESTION: {question}
33 """
```

Key design decisions in this prompt:

- **“ONLY the provided code chunks”** — prevents hallucination
- **“Start with a one-sentence direct answer”** — prevents rambling
- **“Trace execution flow”** — forces the model to connect multiple files
- **“State what’s missing”** — honest about retrieval gaps
- **Explicit formatting rules** — ensures consistent markdown output

The Complete Indexing Pipeline

Step by step, what happens when a repository is first analyzed:

1. Node.js receives POST /api/repo/analyze with repoUrl

2. `simple-git` clones the repository into `temp_repos/repoName/`
3. `fileTree.js` recursively walks the directory, building a JSON tree (ignoring `node_modules`, `.git`, `dist`, etc.)
4. `extractFunctions.js` reads every `.js/.jsx/.ts/.tsx` file, parses it with Babel AST, extracts all functions with name, code, startLine, endLine, type
5. `gitEvolution.js` reads the last 20 git commits, parses diffs, and builds an `evolutionMap` keyed by `filePath::functionName` [NEW]
6. The extracted functions array is sent via HTTP to `POST /index` on the Python service
7. Python service's `vector_store.py` builds enriched chunk text for each function (file path + imports + neighbor names + code)
8. A file summary chunk is prepended for each file
9. All chunk texts are passed to `embed_batch()` — MiniLM encodes them in batches of 64
10. All chunks + embeddings + metadata are stored in ChromaDB under a collection named after the repository
11. Node.js returns `repoName`, `fileCount`, `totalFunctions`, and `tree` to the frontend

The Complete Query Pipeline

Step by step, what happens when a user asks a question:

1. Frontend sends `POST /api/repo/query` with `repoName` and `question`
2. Node.js proxies this to `POST /query` on the Python service
3. Python's `query_collection()` runs **two searches simultaneously**:
 - **Vector search**: Embeds the question with MiniLM, queries ChromaDB for top 20 semantically similar chunks
 - **BM25 search**: Fetches all documents from ChromaDB, builds in-memory BM25 index, scores all chunks against the question tokens, takes top 20
4. RRF merges the two ranked lists into one combined ranking
5. Chunks from test files (`test/`, `tests/`, `examples/`, `.spec.`) are deprioritized
6. Top 5 chunks passed to `ask_llm()` in `llm.py`
7. Gemini 2.5 Flash receives the structured prompt with chunk context
8. LLM returns a structured markdown answer
9. Source files and source functions extracted from chunk metadata
10. Full response returned to Node.js, forwarded to frontend

Node.js Backend: Detailed Breakdown

The backend lives in the `backend/` folder and runs on Express.js port 8000. The backend has been refactored into an MVC structure: routing logic lives in `backend/routes/repo.js` and all business logic has been moved to `backend/controllers/repoController.js`. [NEW]

Key Libraries and Their Roles

Express.js

The web framework. Handles HTTP routing, middleware, request/response lifecycle. All API endpoints are defined here.

simple-git

A Node.js wrapper around Git. Used to clone repositories programmatically:

```
1  await simpleGit().clone(repoUrl, clonePath)
```

This is equivalent to running `git clone` from the command line but within the Node.js process. Cloned repos are stored in `temp_repos/` with the repo name as the folder name. `simple-git` is also used inside `gitEvolution.js` to run `git show` commands for commit diff parsing. [NEW]

@babel/parser and @babel/traverse

These are the AST parsing libraries. `@babel/parser` reads JavaScript/TypeScript source code and produces an Abstract Syntax Tree. `@babel/traverse` walks that tree and calls your callback for each node type.

The parser is configured to handle all modern JS/TS syntax:

```
1  ast = parser.parse(code, {
2    sourceType: 'unambiguous', // handles both ESM and CJS
3    plugins: [
4      'jsx', // React JSX syntax
5      'typescript', // TypeScript types
6      'classProperties', // class fields
7      'optionalChaining', // ?. operator
8      'nullishCoalescingOperator' // ?? operator
9    ],
10   errorRecovery: true // don't crash on syntax errors
11 })
```

axios

HTTP client used by Node.js to call the Python AI service. All communication between Node.js and Python goes through HTTP:

```
1  const response = await axios.post('http://localhost:8001/index', {
2    repo_name: repoName,
3    extracted_functions: extractedFunctions
4  }, { timeout: 300000 })
```

mongoose and MongoDB

MongoDB is used for storing session metadata and repository information. Connected via Mongoose ODM.

cors

Middleware that allows the React frontend (running on port 5173) to make requests to the Express backend (running on port 8000). Without CORS headers, browsers block cross-origin requests.

dotenv

Loads environment variables from `.env` file so sensitive values like `MONGO_URI` are not hard-coded in source code.

API Endpoints

POST /api/repo/analyze

Purpose: Clone a repository, extract functions, compute git evolution, trigger indexing

Body: { `repoUrl`: "https://github.com/..." }

Flow:

1. Validate URL starts with `https://github.com`
2. Derive repo name from URL (`owner/repo` → `owner_repo`)
3. Check if already cloned in `temp_repos/` (cache check)
4. If not cached: `simpleGit().clone(url, path)`
5. Build file tree (ignoring `node_modules`, `.git`, etc.)
6. Count files, reject if over 300
7. Extract functions using Babel AST
8. Compute git evolution map via `gitEvolution.js` [NEW]
9. POST extracted functions to Python `/index`
10. Return {`repoName`, `fileCount`, `totalFunctions`, `tree`} to frontend

POST /api/repo/query

Purpose: Proxy a natural language question to the Python AI service

Body: { `repoName`: "...", `question`: "..." }

Flow:

1. Validate `repoName` and `question` present
2. POST to Python `/query` with `repo_name` and `question`
3. Return Python's response directly to frontend

GET /api/repo/file

Purpose: Read and return the content of a specific file from the cloned repo

Query params: `repoName`, `filePath`

Flow:

1. Construct full path: `temp_repos/repoName/filePath`
2. Check file exists
3. Read file with `fs.readFileSync`
4. Return `{filePath, content, lines, totalLines}`

This endpoint powers both the File Viewer (click a file in the tree) and the Focus Map hover code box.

POST `/api/repo/evolution` [NEW]

Purpose: Return the git change history for functions in the repository, with AI-generated summaries

Body: `{ repoName: "..."` }

Flow:

1. Look up the cloned repo in `temp_repos/repoName/`
2. Call `getCodeEvolution(repoPath, extractedFunctions)` from `gitEvolution.js`
3. Receive `evolutionMap` keyed as `filePath::functionName`
4. POST evolution data to Python `/evolution-summary`
5. Return per-function change history and summaries to frontend

The File Tree Builder (`fileTree.js`)

Recursively walks a directory and returns a JSON tree:

```
1  const IGNORE = new Set([
2    'node_modules', '.git', '.next', 'dist',
3    'build', '.cache', 'coverage'
4  ])
5
6  function buildFileTree(dirPath, relativePath = '') {
7    const entries = fs.readdirSync(dirPath, { withFileTypes: true })
8    const result = []
9
10   for (const entry of entries) {
11     if (IGNORE.has(entry.name)) continue
12     // recursively build children for folders
13     // add file nodes for files
14   }
15   return result
16 }
```

The IGNORE set is critical — without it, `node_modules` alone would produce thousands of nodes and crash the indexing.

The Function Extractor (`extractFunctions.js`)

For each `.js/.jsx/.ts/.tsx` file, this utility:

1. Reads the file content
2. Skips files over 500 KB (likely generated/minified code)
3. Parses with Babel to get AST
4. Traverses AST looking for 4 node types
5. For each function found, stores: `name`, `type`, `code` (sliced from source), `startLine`, `endLine`
6. Returns array of `{filePath, functionCount, functions}`

The Git Evolution Tracker (`gitEvolution.js`) [NEW]

This new utility computes a function-level change history by combining git diff output with the already-extracted function line ranges.

How it works:

1. Reads the last 20 commits from the cloned repo using `simple-git`
2. For each commit, runs `git show -unified=0` to get a minimal diff (no surrounding context lines)
3. Parses the diff to extract: which file was changed, which line numbers were added (+) and deleted (-), and the commit message + timestamp
4. For each changed line, checks which extracted function's `startLine`–`endLine` range it falls within
5. Maps the change to that function, building an `evolutionMap`

The `evolutionMap` structure:

```
1 {  
2   "lib/router/index.js::proto.handle": [  
3     {  
4       commitHash: "a3f9b12",  
5       message:    "fix: handle async errors in middleware",  
6       timestamp:  "2024-11-03T14:22:00Z",  
7       linesAdded: 3,  
8       linesDeleted: 1  
9     },  
10    // ... more commits  
11  ],  
12  "middleware/auth.js::verifyToken": [ ... ]  
13 }
```

This map is then sent to the Python AI service, which asks Gemini to produce a human-readable summary of how each function has evolved over the commits. The summary describes things like “this function was refactored to handle async errors in November and had its JWT logic simplified in December.”

Python AI Service: File-by-File Breakdown

The service lives in the `service/` folder and runs FastAPI on port 8001.

`embedder.py`

Single responsibility: load and expose the MiniLM model. The model loads once at module import time (when FastAPI starts). All other modules import from here.

Key functions:

- `embed_text(text)` — embed a single string (used for queries)
- `embed_batch(texts)` — embed a list of strings efficiently (used during indexing)

`vector_store.py`

All ChromaDB operations are isolated here. Nothing else touches the database directly.

Key functions:

- `get_or_create_collection(repo_name)` — get existing or create new ChromaDB collection
- `is_already_indexed(repo_name)` — check if collection exists and has documents (caching check)
- `delete_collection(repo_name)` — force re-index by deleting existing collection
- `_extract_file_metadata(file_data)` — extract import statements and all function names from a file (used for enriching chunks)
- `_build_chunk_text(file_path, func, file_metadata)` — build the enriched text for one function chunk
- `_build_file_summary_chunk(file_path, file_metadata, file_data)` — build the summary text for one file
- `index_functions(repo_name, extracted_functions)` — main indexing function: builds all chunks, embeds them in batch, stores in ChromaDB
- `query_collection(repo_name, question, top_k)` — hybrid search: BM25 + vector + RRF, returns top chunks
- `_tokenize(text)` — simple regex tokenizer for BM25 (extracts word tokens from code text)
- `_reciprocal_rank_fusion(vector_ranking, bm25_ranking)` — merges two ranked lists using RRF formula

`llm.py`

Single responsibility: call Gemini and return a structured response.

Key function: `ask_llm(question, context_chunks)`

Takes the question and retrieved chunks, builds the structured prompt, calls Gemini, extracts `source_files` and `source_functions` from chunk metadata, returns everything.

The `client` object is also exported and reused by `main.py` for the evolution summary endpoint, so Gemini is only initialized once. **[NEW]**

`main.py`

FastAPI entry point. Routes only — no business logic here.

Endpoints:

- GET `/health` — health check
- POST `/index` — receive extracted functions from Node.js, index them
- POST `/query` — receive question + repo name, return answer + sources
- DELETE `/index/{repo_name}` — force delete collection for re-indexing
- POST `/evolution-summary` — receive evolutionMap from Node.js, ask Gemini to summarize per-function change history, return summaries **[NEW]**

Frontend: React Application

The frontend lives in `frontend/` and runs on Vite + React, port 5173.

Technology Stack

- **React 18** — component-based UI
- **Vite** — build tool and dev server (faster than Create React App)
- **React Router v6** — client-side navigation between pages
- **Tailwind CSS v3** — utility-first styling
- **axios** — HTTP client for API calls
- **react-markdown** — renders LLM responses as formatted markdown

Pages

LandingPage.jsx

The entry point. Contains:

- Hero section with headline and subheadline
- GitHub URL input bar with Analyze button
- Step-by-step loading progress panel (appears while analyzing)
- Three feature cards (Zero Setup, AI Q&A, Focus Map)
- How it works section with 3 numbered steps

On clicking Analyze:

1. Calls `POST /api/repo/analyze` with 5-minute timeout (cloning + indexing takes time)
2. Shows loading panel with pipeline steps
3. On success: navigates to `/chat` passing `repoName`, `fileCount`, `totalFunctions`, `tree` via React Router state
4. On error: shows error message inline

ChatPage.jsx

The main application view. Split layout — left 60%, right 40%.

Left panel:

- Top bar with repo name, file count badge, functions count badge, back button
- Scrollable message history
- Bouncing dots loading indicator while waiting for AI
- Fixed input bar at bottom (textarea + send button)
- Enter key submits, Shift+Enter for newline

Right panel (3 tabs):

- **File Tree** — collapsible folder/file navigator
- **Focus Map** — visual map of relevant files and functions
- **File Viewer** — code viewer with line numbers

State management:

```
1  const [messages, setMessages]      = useState([...]) // chat history
2  const [question, setQuestion]      = useState('')    // input value
3  const [loading, setLoading]        = useState(false)  // waiting for AI
4  const [activeTab, setActiveTab]     = useState('filetree') // right panel
5  const [focusData, setFocusData]   = useState(null)   // focus map data
6  const [selectedFile, setSelectedFile] = useState(null) // file viewer
```

Components

ChatMessage.jsx

Renders a single chat bubble. User messages are right-aligned with indigo background. AI messages are left-aligned with dark background.

AI messages use `react-markdown` with custom component overrides to render:

- **bold** as white bold text
- Bullet lists with proper spacing
- Numbered lists for sequential steps
- Inline code (`'code'`) with purple background highlight

Each AI message with `sourceFunctions.length > 0` shows a "View in Focus Map →" button. Clicking it calls `onViewFocusMap(sourceFunctions, sourceFiles)`, which updates the right panel.

FileTree.jsx

Recursive tree component. Each node is either a folder (clickable to expand/collapse) or a file (clickable to open in File Viewer).

```
1  function TreeNode({ node, onFileClick }) {
2    const [open, setOpen] = useState(true)
3
4    if (node.type === 'file') {
5      return <div onClick={() => onFileClick(node.path)}>...</div>
6    }
7    // folder with collapsible children
8  }
```

Clicking a file calls `onFileClick(filePath)`, which sets `selectedFile` in `ChatPage` and switches to the File Viewer tab.

FileViewer.jsx

Fetches and displays file content with line numbers. Uses `GET /api/repo/file` endpoint.

Renders as an HTML table: left column is line numbers (non-selectable), right column is code with `whitespace-pre` to preserve indentation.

FocusMap.jsx

Groups `source_functions` by file. Renders each file as a container node with its functions as child nodes inside.

Each function node (`FunctionNode` component):

- Shows function name and line number
- On hover: fetches file content from `/api/repo/file`, slices the relevant lines using `startLine` and `endLine`, shows a fixed-position tooltip with the code
- Tooltip uses `position: fixed` with JavaScript-calculated coordinates to avoid being clipped by overflow containers
- Smart positioning: detects available screen space and shows tooltip on left or right accordingly

Filters out `__file_summary__` nodes (these are metadata, not real functions). Also filters out functions with the same `__file_summary__` marker from the focus map display.

RepoHeader.jsx

Displays repository metadata in the chat page header. Shows the repository name, file count, and total functions count as styled badges. Previously a placeholder, this component is now fully implemented. **[UPDATED]**

Phase-Wise Development: What Was Built, Issues Found, How Fixed

Phase 1: Core Pipeline — Clone, Parse, Index

What was built

1. **POST /api/repo/analyze endpoint** — accepts GitHub URL, clones with simple-git, builds file tree
2. **fileTree.js** — recursive directory walker with ignore list
3. **extractFunctions.js** — Babel AST parser for JS/TS function extraction
4. **Python /index endpoint** — receives functions, embeds with MiniLM, stores in ChromaDB
5. **Python /query endpoint** — basic vector-only search, sends to Gemini, returns answer
6. **POST /api/repo/query in Node.js** — proxy to Python

Issues Found

Issue 1: Port 5000 blocked on Mac

macOS Monterey and later reserves port 5000 for AirPlay Receiver. Every request to port 5000 got a 403 Forbidden — never reached Express. Fixed by switching to port 8000.

Issue 2: Module system mismatch

Initial code mixed ES modules (`import/export`) with CommonJS (`require/module.exports`). Node.js cannot mix these. Fixed by standardizing all backend files to CommonJS.

Issue 3: ChromaDB include parameter

ChromaDB's `query()` method does not accept "ids" in the include list (IDs are always returned automatically). Caused `ValueError` on every query. Fixed by removing "ids" from the include list.

Issue 4: Gemini SDK deprecation

The `google-generativeai` Python package was deprecated by Google. Updated to `google-genai` and changed the API call pattern to:

```
1 client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))
2 response = client.models.generate_content(
3     model="gemini-2.5-flash",
4     contents=prompt
5 )
```

Note: During environment setup (Phase 6), the virtual environment was found to have `google-generativeai` installed rather than `google-genai`. The code was temporarily reverted to `google-generativeai` to match the installed package. Migration back to `google-genai` is pending. A `FutureWarning` is emitted at service startup but does not affect functionality. **[UPDATED]**

Issue 5: numpy binary incompatibility on Mac

After upgrading numpy, scikit-learn became incompatible (`numpy.dtype` size changed). Fixed by: `pip uninstall numpy scikit-learn -y` and `pip install numpy scikit-learn`.

Issue 6: Sending 22,000 lines of JSON via Postman

Attempted to manually POST the extracted functions to the Python service from Postman — impossible due to size. Fixed by wiring Node.js to automatically call Python's `/index` after extraction (server-to-server call with 5 minute timeout).

Result at End of Phase 1

Full pipeline working: paste GitHub URL → functions extracted → indexed in ChromaDB → questions answered via Postman. 3093 functions from `expressjs/express` indexed in ~30 seconds.

Phase 2: RAG Quality Improvements

Problem Identified

The initial RAG worked but answers were poor quality. Root causes:

1. Raw function code has no context — the embedding model doesn't know which file or module the function belongs to
2. 70% of functions are named “anonymous” — no semantic signal in the chunk text
3. Pure vector search misses exact keyword matches
4. Test files dominated retrieval results — `test/` folder had 80+ files in Express repo, overwhelming source files

Upgrade 1: Enriched Chunks

Built `_build_chunk_text()` in `vector_store.py` that prepends file path, folder context, imports, and sibling function names to each chunk before embedding. This gave the embedding model rich semantic context. Retrieval quality improved significantly — source files started appearing in results instead of only test files.

Upgrade 2: File Summary Chunks

Added `_build_file_summary_chunk()` that generates one holistic summary per file. For a 200-file repo, this adds 200 extra chunks that answer architectural questions. Total chunks went from 3093 to 3227 for Express.

Upgrade 3: Hybrid Search (BM25 + Vector + RRF)

Installed `rank-bm25` library. Rewrote `query_collection()` to run both searches in parallel, then merge with RRF. This solved the exact keyword matching problem — queries with specific function names, file names, or error codes now reliably retrieve the right chunks.

Test File Filtering

Added a filter in `main.py` to deprioritize chunks from `test/`, `tests/`, `examples/`, `.spec.`, `.test.` paths. If filtered results are empty, falls back to original results.

Result at End of Phase 2

Noticeably better retrieval. Auth questions return `controller/authController.js` instead of test files. File summary chunks appear in Focus Map for architectural questions.

Phase 3: Prompt Engineering

Problem Identified

The LLM was receiving good chunks but producing generic answers like “Authentication appears to involve issuing a token.” Not useful for a developer who needs to understand the system.

Solution

Rewrote the prompt with explicit structure:

- **Role definition** — expert code navigator
- **Step-by-step answering instructions** — one-sentence answer, then details, trace flow, cite line numbers
- **Formatting rules** — bold for file names, bullet points, numbered steps for flows
- **Strict hallucination prevention** — only state what is clearly visible in chunks
- **Gap reporting** — explicitly state what information is missing when chunks are insufficient

Result

Answer quality dramatically improved. Responses now include numbered sections, bold citations, execution flow tracing, and a “What is missing” section when chunks don’t fully cover the question. This “missing” section is particularly valuable — it’s honest rather than hallucinating.

Phase 4: Frontend Development

Landing Page

Built with React + Tailwind CSS. Dark theme (#0a0a0a background, white text, indigo accent). Contains URL input, loading state panel, feature cards, how-it-works section.

Chat Page

Split layout with three-tab right panel. Connected to real backend endpoints. Markdown rendering via `react-markdown`. Bouncing dots loading indicator.

File Tree Component

Recursive tree with expand/collapse folders. Clicking files triggers File Viewer tab.

File Viewer Component

Fetches file content from `GET /api/repo/file`. Renders as table with line numbers on left column. Preserves indentation with `whitespace-pre`.

Focus Map Component

Initially just showed a text list of source files. Upgraded to grouped node layout (file as container, functions as child nodes). Added hover tooltip with code preview using fixed positioning to avoid overflow clipping issues.

Issues During Frontend Development

Issue 1: All component files empty

`touch` command creates empty files. VS Code showed the file path but contents were empty. Every import caused “does not provide default export” error. Fixed by manually pasting and saving each component.

Issue 2: Tailwind v4 incompatibility

`npx tailwindcss init -p` fails on Tailwind v4 (no `init` command). Fixed by uninstalling v4 and installing `tailwindcss@3` which still has the `init` command.

Issue 3: Hover tooltip clipped

The code preview tooltip was positioned `absolute` inside an overflow-hidden container. Tooltip was invisible (clipped). Fixed by switching to `position: fixed` with JavaScript-calculated screen coordinates, keeping the tooltip in the viewport layer above all containers.

Issue 4: Tooltip going off-screen

Fixed-position tooltip appeared off the right edge of the screen. Fixed by checking `window.innerWidth - rect.right` (available space on right) and showing tooltip on the left side when insufficient space exists on the right.

Phase 5: GitHub and Documentation

1. Created `.gitignore` to exclude `node_modules`, `temp_repos`, `chroma_store`, `.env` files
2. Initialized git repository
3. Pushed to `github.com/Sahil0282/repo-explorer`
4. Added comprehensive README with architecture diagram, setup instructions, tech stack explanation
5. Added RepoExplorer as first project on resume with 6 bullet points covering the technically impressive aspects

Phase 6: Git Evolution Feature [NEW]

Motivation

After completing the core RAG Q&A pipeline, a second feature was added to answer a fundamentally different type of question: not “how does this code work?” but “how has this code changed over time?” This is useful for understanding why a function is structured the way it is, or for identifying which functions have been heavily modified (indicating instability or active development).

What Was Built

1. `gitEvolution.js` in `backend/utils/` — reads the last 20 commits using `simple-git`, runs `git show -unified=0` per commit, parses added/deleted line ranges, and maps them to extracted functions by line range overlap
2. `POST /api/repo/evolution` in Node.js — new endpoint that triggers the evolution computation and proxies results to the Python service
3. `POST /evolution-summary` in Python `main.py` — new FastAPI endpoint that receives the `evolutionMap` and uses the shared Gemini client to generate per-function change summaries

4. **MVC refactor** — took the opportunity to separate routing from business logic. All controller functions moved to `backend/controllers/repoController.js`; `backend/routes/repo.js` now only defines routes

Issues During Phase 6

Issue 1: Wrong uvicorn launch path

Running `uvicorn main:app` from the repo root failed because `main.py` is inside `service/`. Fixed by `cd service` first, then `uvicorn main:app --reload --port 8001`.

Issue 2: rank-bm25 not in requirements.txt

The `rank-bm25` package was added to `vector_store.py` during Phase 2 but was never added to `requirements.txt`. Service failed to start with `ModuleNotFoundError: No module named 'rank_bm25'`. Fixed by `pip install rank-bm25`. The `requirements.txt` file must be regenerated with `pip freeze` to capture this dependency permanently.

Issue 3: Gemini SDK mismatch

`llm.py` was importing from `google import genai` (the new `google-genai` style), but the virtual environment had `google-generativeai` installed (the old package). Caused `ImportError` at startup. Temporarily resolved by reverting `llm.py` to use `google.generativeai`. Full migration to `google-genai` is a known pending task.

Result at End of Phase 6

All three services start cleanly. The evolution pipeline is operational: pasting a repo URL triggers both the RAG indexing pipeline and the git evolution computation in a single `/analyze` call. Per-function change summaries are generated by Gemini and available via the `/evolution` endpoint.

Current Limitations of the MVP

Repository Constraints

- **Public repos only** — private repositories require GitHub OAuth token integration
- **JS/TS only** — Python, Java, Go, Rust files are not parsed
- **Under 300 files** — larger repos are rejected to control embedding cost and time
- **Under 50 MB** — very large files (generated/minified code) are skipped

RAG Quality Limitations

- **Anonymous functions** — 60–70% of functions have no name (callbacks, arrow functions, IIFE). These are labeled `anonymous` or `fn @ L{line}` which provides poor semantic signal for retrieval
- **No call graph** — the system doesn't know which function calls which. It can't trace execution flows across files unless the LLM infers it from the chunks
- **Semantic gap for framework-specific terminology** — Express internally calls middleware “next”, not “middleware”. Queries using the domain term don't match the implementation term
- **Test file dominance** — repositories with many test files (like Express with 80+ test files) have test chunks outnumbering source chunks, biasing retrieval

Infrastructure Limitations

- **No deployment** — currently localhost only; requires all three services running locally
- **No session persistence** — refreshing the chat page loses all conversation history
- **No rate limiting** — a malicious user could trigger unlimited indexing jobs
- **BM25 rebuilt on every query** — fetches all ChromaDB documents and rebuilds BM25 index in memory for each query (slow for large repos; should be cached)
- **No auth** — anyone who knows the localhost URL can use it
- **requirements.txt out of sync** — `rank-bm25` was installed manually and not yet added to `requirements.txt`. A fresh `pip install -r requirements.txt` will fail until the file is regenerated with `pip freeze [NEW]`
- **Gemini SDK FutureWarning** — `llm.py` currently uses `google-generativeai` (deprecated) instead of the newer `google-genai`. Service works but emits a warning at startup [NEW]
- **Git evolution only covers last 20 commits** — repositories with long histories will have incomplete evolution data [NEW]

Ongoing and Future Improvements

Improvement 1: Better Anonymous Function Naming (High Priority)

Problem: Most Express route handlers look like:

```
1 app.get('/users', async (req, res) => {
2   // handler code
3 })
```

This is an arrow function with no name — stored as `anonymous`. But from context, we know its name should be `GET /users route handler`.

Solution: During AST traversal in `extractFunctions.js`, when an anonymous function is encountered, look at its parent node. If the parent is:

- `app.get('/path', fn)` → name it `GET /path handler`
- `router.post('/path', fn)` → name it `POST /path handler`
- `app.use(middleware)` → name it `middleware: {fn body preview}`
- `mongoose.Schema({...})` → name it `Schema definition`
- `Promise.then(fn)` → name it `then callback`

This dramatically improves both the focus map labels and the retrieval quality because the chunk text now says “GET /users route handler” instead of “anonymous”.

Implementation: In `ArrowFunctionExpression` handler in `extractFunctions.js`, inspect `path.parent` and `path.parentPath.parent` to determine context.

Improvement 2: Python File Support

Problem: The AST parser only handles JS/TS. Python repositories (which are common) cannot be indexed.

Solution: Add a Python parser in the Python AI service using the `ast` standard library module:

```
1 import ast
2
3 def extract_python_functions(file_path: str):
4     with open(file_path) as f:
5         tree = ast.parse(f.read())
6
7     functions = []
8     for node in ast.walk(tree):
9         if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
10             functions.append({
11                 "name": node.name,
12                 "startLine": node.lineno,
13                 "endLine": node.end_lineno,
14                 "code": ast.get_source_segment(source, node)
15             })
16     return functions
```

The Node.js backend can detect `.py` files and forward them to a new `/parse-python` endpoint in the Python service, which returns the extracted functions in the same format as the JS extractor.

Improvement 3: Call Graph / Dependency Tracking

Problem: Chunks are isolated — the system doesn't know that `isAuth.js` middleware is called before `productController.js`.

Solution: During AST traversal, record:

1. Which functions each file imports from other files
2. Which functions each function calls internally
3. Which routes use which middleware (from `router.use()` calls)

Store this as a call graph in MongoDB. When a query retrieves chunk A, also fetch chunks that A imports or that import A (one hop expansion). This gives the LLM connected context rather than isolated fragments.

Improvement 4: BM25 Index Caching

Problem: Currently, every query fetches all ChromaDB documents and rebuilds the BM25 index from scratch. For a 3000-chunk repository, this adds ~200ms overhead per query.

Solution: Cache the BM25 index in memory keyed by repo name. Only rebuild when a new version of the repo is indexed:

```
1  bm25_cache = {}  # module-level cache
2
3  def get_bm25_index(repo_name, documents, ids):
4      if repo_name not in bm25_cache:
5          tokenized = [_tokenize(doc) for doc in documents]
6          bm25_cache[repo_name] = {
7              "index": BM25Okapi(tokenized),
8              "ids": ids
9          }
10     return bm25_cache[repo_name]
```

Improvement 5: Session Persistence

Problem: Refreshing the chat page loses all conversation history and the current repo context.

Solution: Store chat sessions in MongoDB. On page load, check for an existing session with the current `repoName` in `localStorage`. If found, fetch the session history from the backend and restore the state.

Add a MongoDB schema:

```
1  const SessionSchema = new Schema({
2      repoName: String,
3      messages: [{ role: String, text: String,
4                    sourceFiles: [String],
5                    sourceFunctions: [Object] }],
6      createdAt: { type: Date, expires: '24h' } // auto-delete after 24h
7  })
```

Improvement 6: Deployment

Target architecture:

- **Frontend:** Vercel (free tier, automatic CI/CD from GitHub)
- **Node.js Backend:** Render (free tier, Docker container)
- **Python AI Service:** Render (free tier, Docker container with `sentence-transformers` and `chromadb` installed)

Challenges for deployment:

- `temp_repos/` needs persistent disk storage (Render provides this)
- `chroma_store/` needs persistent disk storage (same)
- MiniLM model download on first boot takes time — need to pre-bake into Docker image
- Free tier services sleep after inactivity — need a keep-alive ping

Improvement 7: Clickable File Paths in Chat

Problem: The LLM response mentions files like `controller/authController.js` as plain text. The user has to manually click the File Tree to open that file.

Solution: Post-process the markdown answer in `ChatMessage.jsx` using a regex to detect file path patterns (`[a-zA-Z0-9/_.-]+\.(js|ts|jsx|tsx|py)`) and wrap them in clickable elements that open the File Viewer directly:

```
1 // In custom markdown renderer
2 code: ({ children }) => {
3   const filePath = String(children)
4   if (isFilePath(filePath)) {
5     return <span
6       onClick={() => onFileClick(filePath)}
7       className="text-indigo-400 cursor-pointer hover:underline"
8     >{children}</span>
9   }
10  return <code>...</code>
11 }
```

Improvement 8: Rate Limiting

Problem: No protection against abuse. A user could analyze hundreds of repos, consuming disk space and API credits.

Solution: Add rate limiting middleware to the Node.js backend:

```
1 const rateLimit = require('express-rate-limit')
2
3 const analyzeLimiter = rateLimit({
4   windowMs: 60 * 60 * 1000, // 1 hour
5   max: 5, // max 5 repos per IP per hour
6   message: { error: 'Too many requests. Try again later.' }
7 })
```

```
8
9  const queryLimiter = rateLimit({
10      windowMs: 60 * 1000, // 1 minute
11      max: 20, // max 20 questions per minute per IP
12  })
13
14  router.post('/analyze', analyzeLimiter, analyzeRepo)
15  router.post('/query', queryLimiter, queryRepo)
```

Improvement 9: Migrate Gemini SDK to google-genai [NEW]

Problem: The service currently uses `google-generativeai`, which Google has deprecated. A `FutureWarning` is emitted at every startup. The newer `google-genai` package offers a cleaner client API and will be the only supported option going forward.

Solution:

```
1  pip uninstall google-generativeai
2  pip install google-genai
3  pip freeze > requirements.txt
```

Then update `llm.py`:

```
1  from google import genai
2
3  client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))
4
5  response = client.models.generate_content(
6      model="gemini-2.5-flash",
7      contents=prompt
8  )
9  answer = response.text
```

Improvement 10: Fix requirements.txt [NEW]

Problem: `rank-bm25` was installed manually and is missing from `requirements.txt`. A fresh environment setup will fail.

Solution: After completing the Gemini SDK migration above, regenerate the file:

```
1  cd service
2  source ../.venv/bin/activate
3  pip freeze > requirements.txt
```

Verify the output contains at minimum: `rank-bm25`, `google-genai`, `fastapi`, `uvicorn`, `sentence-transformers`, `chromadb`, `numpy`.

Interview Preparation: Questions You Will Be Asked

System Design Questions

Q: Why did you separate the backend into Node.js and Python instead of using one language?

A: Node.js has excellent tooling for the web layer — serving HTTP, routing, file system operations, git cloning. Python has far better ML ecosystem support — sentence-transformers, ChromaDB, BM25, LangChain all have native Python libraries. Using Python for the AI service and Node.js for the web layer lets each do what it's best at. They communicate via HTTP.

Q: Why not just send the entire codebase to the LLM context?

A: Two reasons. First, a codebase with 3000+ functions would exceed the context window of any LLM. Second, even if it fit, attention dilution becomes a problem — the LLM performs worse with irrelevant context. RAG lets us send only the 5–8 most relevant chunks, giving the LLM focused context and better answers.

Q: How does your hybrid search work?

A: We run two searches simultaneously. BM25 scores all chunks based on exact keyword frequency and inverse document frequency. Vector search embeds the query and finds semantically similar chunks using cosine similarity. We merge the two ranked lists using Reciprocal Rank Fusion: $\text{score}(d) = \frac{1}{60 + \text{rank}_1(d)} + \frac{1}{60 + \text{rank}_2(d)}$. Documents appearing high in both lists score highest.

Q: What is the caching strategy?

A: Two levels. First, cloned repositories are stored in `temp_repos/` and never re-cloned if already present. Second, ChromaDB collections persist to disk in `chroma_store/`. The `is_already_indexed()` function checks if a collection exists before triggering re-embedding. A repository is indexed once and served forever.

Q: You have two separate AI pipelines. How are they different? [NEW]

A: Yes, the system has two completely independent pipelines. The RAG Q&A pipeline indexes code chunks (function bodies and file summaries) as MiniLM embeddings in ChromaDB, and uses hybrid BM25 + vector search to retrieve relevant chunks for each user question. The git evolution pipeline is entirely separate — it uses `git show` to get commit diffs, maps line changes to function line ranges to identify which functions changed in which commits, and sends that structured history to Gemini as plain text for summarization. Git diffs are never embedded; they go directly to the LLM.

AI/ML Questions

Q: Why MiniLM over OpenAI embeddings?

A: MiniLM is free and runs locally. OpenAI charges per token for embeddings. For a student project indexing thousands of functions across multiple repositories, OpenAI embeddings would incur real cost. MiniLM gives 80–90% of the quality at zero cost.

Q: What is the dimensionality of your embeddings and why does it matter?

A: MiniLM produces 384-dimensional vectors. Higher dimensionality (like OpenAI's 1536) captures more nuance but requires more storage and computation. 384 dimensions is sufficient for our use case — we're doing code retrieval, not fine-grained semantic similarity for medical or legal documents.

Q: How do you prevent hallucination?

A: Three mechanisms. First, the prompt strictly says “answer ONLY based on the provided chunks.” Second, it says “only state what is clearly visible.” Third, it instructs the LLM to explicitly report what information is missing if the chunks don't fully cover the question. This makes the LLM honest about its limitations rather than filling gaps with invented information.

Q: What is Reciprocal Rank Fusion and why use it over score averaging?

A: RRF merges ranked lists based on position, not score. Score averaging fails because BM25 and vector search produce scores on completely different scales — BM25 might score 45.2 while cosine similarity scores 0.87. Normalizing them introduces arbitrary biases. RRF only uses rank position (1st, 2nd, 3rd...) which is scale-independent. A document ranked 1st in both lists always wins, regardless of the absolute scores.

System-Specific Questions

Q: How does your AST parser handle different function types?

A: We handle four types: FunctionDeclaration (`function foo()`), FunctionExpression (`const foo = function()`), ArrowFunctionExpression (`const foo = () =>`), and ClassMethod (methods in ES6 classes). The Babel traverse API calls our callback for each node type.

Q: What happens when someone asks about code that's not in the top 5 chunks?

A: The LLM is instructed to explicitly state what is missing. Instead of hallucinating, it says “Based on the provided chunks, I can see X and Y. However, the chunks don't show how Z works. To understand Z, you should look at [files].” This is more useful than a confident but wrong answer.

Q: How do you handle the focus map if all functions are anonymous?

A: We display them as `filename.js:lineNumber` instead of function names. This at least tells the user where in the file to look. The proper fix (in progress) is to infer names from context — route handlers become “GET /path handler”, etc.

Q: How does the git evolution feature work technically? [NEW]

A: `gitEvolution.js` iterates over the last 20 commits using `simple-git`. For each commit, it runs `git show -unified=0` which returns a diff with zero surrounding context lines, making it easy to extract exactly which line numbers changed. For each changed line, it checks whether that line falls within any extracted function's `startLine`–`endLine` range. If it does, that commit is associated with that function. The result is an `evolutionMap` where each key is `filePath::functionName` and the value is a list of commit objects. This map is sent to Gemini, which produces a natural-language summary of how the function evolved.

Complete Tech Stack Reference

Category	Technology	Purpose
<i>Frontend</i>		
Framework	React 18	Component-based UI
Build Tool	Vite	Dev server and bundling
Routing	React Router v6	Client-side navigation
Styling	Tailwind CSS v3	Utility-first CSS
HTTP Client	axios	API calls to backend
Markdown	react-markdown	Render LLM responses
<i>Node.js Backend</i>		
Framework	Express.js	HTTP server and routing
Git Ops	simple-git	Clone repos; run git show for evolution [UPDATED]
JS/TS Parser	@babel/parser	Parse code to AST
AST Walker	@babel/traverse	Walk AST nodes
HTTP Client	axios	Call Python service
Database	MongoDB + Mongoose	Session and repo metadata
Environment	dotenv	Load .env variables
CORS	cors	Allow cross-origin requests
Git History	gitEvolution.js	Map commit diffs to functions [NEW]
MVC	repoController.js	Separated business logic from routing [NEW]
<i>Python AI Service</i>		
Framework	FastAPI	Python HTTP service
Embed Model	MiniLM-L6-v2	Text to vector (384-dim)
Embed Lib	sentence-transformers	Run MiniLM locally
Vector DB	ChromaDB	Store and search embeddings
Keyword	rank-bm25	BM25 keyword retrieval
Merge	RRF (custom)	Combine BM25 + vector
LLM	Gemini 2.5 Flash	Answer generation + evolution summaries [UPDATED]
LLM SDK	google-generativeai	Python client for Gemini (migration to google-genai pending)
Numerical	numpy	Vector operations
<i>Infrastructure</i>		
Version Ctrl	Git + GitHub	Source control
Runtime (BE)	Node.js	JavaScript server runtime
Runtime (AI)	Python 3.11	AI service runtime

Repository: github.com/Sahil0282/repo-explorer

Author: Sahil Arun Dhawane | Vishwakarma Institute of Technology, Pune

Program: B.Tech Electronics & Telecommunications (*Expected 2027*)